

HW05 实验报告：极简 CNN 与 LeNet-5 实现与对比

1. 文章学习摘要

本次实验基于经典 MNIST 手写数字识别数据集，完成了极简卷积神经网络（Simple CNN）与经典 LeNet-5 架构的实现与训练。

核心学习要点：

- ****CNN 核心原理****：理解卷积层（提取特征）、池化层（降维采样）与全连接层（分类器）的功能分工。
- ****LeNet-5 历史意义****：作为深度学习领域的奠基性模型，其结构为后续深度神经网络提供了经典范式（Conv + Pool + FC）。
- ****训练流程****：掌握了基于 PyTorch 的数据加载、前向传播、反向传播与模型评估的完整生命周期。

2. LeNet-5 结构说明

2.1 原始 LeNet-5 结构

LeNet-5 由 Yann LeCun 等人在 1998 年提出，最初用于识别银行支票。

- ****输入层****： 32×32 灰度图像。
- ****C1（卷积层）****：6 个 5×5 卷积核，输出 $6 @ 28 \times 28$ 。
- ****S2（池化层）****： 2×2 最大池化，步长 2，输出 $6 @ 14 \times 14$ 。
- ****C3（卷积层）****：16 个 5×5 卷积核，输出 $16 @ 10 \times 10$ 。
- ****S4（池化层）****： 2×2 最大池化，步长 2，输出 $16 @ 5 \times 5$ 。
- ****F5（全连接层）****：120 个神经元。
- ****F6（全连接层）****：84 个神经元。
- ****输出层****：10 个神经元（对应 0-9 数字分类）。

2.2 本次实现结构（PyTorch 版）

本次实现严格遵循经典架构，仅适配现代 PyTorch 语法：

```
```python
```

# 核心结构对应

```
self.conv1 = nn.Conv2d(1, 6, 5) # C1
self.pool = nn.MaxPool2d(2, 2) # S2/S4
self.conv2 = nn.Conv2d(6, 16, 5) # C3
self.fc1 = nn.Linear(16*5*5, 120) # F5
self.fc2 = nn.Linear(120, 84) # F6
self.fc3 = nn.Linear(84, 10) # 输出
```

```
Windows PowerShell
版权所有 (C) Microsoft Corporation. 保留所有权利。

安装最新的 PowerShell，了解新功能和改进！ https://aka.ms/PSWindows

PS C:\Users\asus> cd Desktop/hw05
PS C:\Users\asus\Desktop\hw05> python lenet_train.py
Traceback (most recent call last):
 File "C:\Users\asus\Desktop\hw05\lenet_train.py", line 55, in <module>
 train(epoch)
    ~~~~~^~~~~~
  File "C:\Users\asus\Desktop\hw05\lenet_train.py", line 35, in train
    loss = F.cross_entropy(output, target)
  File "C:\Users\asus\AppData\Local\Programs\Python\Python313\Lib\site-packages\torch\nn\functional.py", line 3504, in cross_entropy
    return torch._C._nn.cross_entropy_loss(
           ~~~~~^~~~~~
 input,
        ~~~~~^
        ...<5 lines>...
        label_smoothing,
        ~~~~~^~~~~~
)
 ^
ValueError: Expected input batch_size (100) to match target batch_size (64).
PS C:\Users\asus\Desktop\hw05> python lenet_train.py
Epoch 1 训练完成
LeNet-5 测试准确率：97.87%
Epoch 2 训练完成
LeNet-5 测试准确率：98.40%
Epoch 3 训练完成
LeNet-5 测试准确率：98.78%
Epoch 4 训练完成
LeNet-5 测试准确率：99.01%
Epoch 5 训练完成
LeNet-5 测试准确率：98.79%
PS C:\Users\asus\Desktop\hw05>
```

```
Windows PowerShell
Requirement already satisfied: setuptools<82 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from torch) (81.0.0)
Requirement already satisfied: sympy>=1.13.3 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from torch) (3.6.1)
Requirement already satisfied: Jinja2 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from torch) (2026.3.0)
Requirement already satisfied: numpy in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from torchvision) (2.2.6)
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from torchvision) (11.3.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from Jinja2->torch) (3.0.3)
Downloading torchvision-0.26.0-cp313-cp313-win_amd64.whl (4.3 MB)
4.3/4.3 MB 1.6 MB/s 0:00:02
Installing collected packages: torchvision
Successfully installed torchvision-0.26.0
PS C:\Users\asus> cd Desktop\hw05
PS C:\Users\asus\Desktop\hw05> python train.py
100% | 9.91M/9.91M [00:11<00:00, 880kB/s]
100% | 28.9k/28.9k [00:00<00:00, 114kB/s]
100% | 1.65M/1.65M [00:02<00:00, 754kB/s]
100% | 4.54k/4.54k [00:00<00:00, 17.1MB/s]
Epoch 1 训练完成
测试准确率: 97.46%
Epoch 2 训练完成
测试准确率: 98.35%
Epoch 3 训练完成
测试准确率: 98.35%
Epoch 4 训练完成
测试准确率: 98.48%
Epoch 5 训练完成
测试准确率: 98.59%
PS C:\Users\asus\Desktop\hw05>
```

### 3.1 训练配置（超参数）

优化器：Adam

学习率（lr）：0.001

批量大小（batch\_size）：64

训练轮数（epochs）：5

损失函数：交叉熵损失（CrossEntropyLoss）

设备：GPU（若可用） / CPU

### 4. 极简 CNN 与 LeNet-5 对比分析

#### 1. 结构复杂度：

极简 CNN：仅 1 层卷积 + 1 层池化，适合快速验证或资源受限场景。

LeNet-5：2 层卷积 + 3 层全连接，属于标准深度 CNN 结构，特征学习能力更强。

2. 准确率：LeNet-5 凭借更深的网络结构，在 MNIST 数据集上取得了略高的测试准确率。

3. 适用场景：极简 CNN 适合快速原型开发；LeNet-5 适合学习经典 CNN 设计范式，是理解复杂网络的基础。

#### 5. 参考与引用

PyTorch 官方文档：<https://pytorch.org/docs/stable/index.html>

MNIST 数据集：<http://yann.lecun.com/exdb/mnist/>

LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition.

模型 测试集准确率 模型参数量量级

特点

极简 CNN ~98.5% 约 30k 结构极简，仅含一层卷积，训练速度快，满足基础分类需求。

LeNet-5 ~99.0% 约 60k 经典深度架构，特征提取层次更深，泛化能力略强。